

Cartera: el pool, el universo, las parejas y la búsqueda de fondeo

AlgoProject — manual de uso. Actualizado el 2026-09-30 con `tools/manual.py`.

1. 77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo archivarlas
2. 78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas
3. 79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan

77. Cartera · el pool de desarrollo — qué estrategias pueden entrar y cómo

archivarlas

Qué pregunta responde

¿Qué estrategias de tus proyectos pueden entrar en el pool con el que se desarrolla el motor de carteras, y con qué orden exacto se archiva cada una? El comando lee los veredictos que ya dejaron los estudios de cada paso y te imprime, estrategia por estrategia, la línea de `core.archive` que la congela. **No archiva nada**: tú apruebas la lista y lanzas las líneas que quieras.

Es el hito M0 del plan de carteras (`portfolio/PLAN.md`). Las carteras de este pool **no son operables**: son para desarrollar y probar el motor hasta que exista el pool validado del paso 26.

Cuándo lo usas, y cuándo no

Úsalo cuando un run del workflow ha terminado y quieres llevar sus estrategias al archivo para construir carteras con ellas. **No** lo uses mientras SQX está corriendo ese proyecto: la lista sale igual, pero el archivo se niega a congelar una estrategia de un proyecto que SQX tiene en marcha.

Antes de empezar

Nada. Solo lee `AlgoData/reports/` y `AlgoData/strategyPermutations/`; no toca SQX ni el demonio.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.candidates --project Test_USDJPY_donchianUpperCrossUp_H1
```

flag	obligatorio	qué hace
<code>--project</code>	no	un solo proyecto; sin él, todos los que tienen informes

Tarda unos segundos. No toca SQX.

Qué produce

Nada en disco: solo imprime. Por proyecto, una línea por candidata con su clase, su paso, sus palabras y la orden de archivo, y al final cuántas quedaron fuera y por qué palabra.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Superviviente**: la palabra del último paso que la juzgó es MANTENER, survives, worth_it, predicts o PASS.
- **Dudosa** (casi-superviviente): DUDOSA o INCONCLUSIVE. Entra al pool, marcada.
- **Lo que el workflow llevó adelante manda**: toda madre con lote de variantes en el 16.5 entra como superviviente del paso 16.5, digan lo que digan las palabras de antes. Su línea muestra **todas** esas palabras (también los DESCARTAR) para que lo veas al aprobarla.
- El FAIL de **mcRetest** no excluye: informa, no filtra (así lo hizo el workflow).
- Una estrategia ya archivada no se repite.

Una estrategia archivada con paso < 26 es **de desarrollo** automáticamente; no hay que marcarla.

Un ejemplo completo

Salida real del 2026-09-30, con el run H1 aún en el paso 16.5 (recortada: la lista entera tiene 170 líneas de candidatas):

```
=== Test_USDJPY_donchianUpperCrossUp_H1 ===
  [SUPERVIVIENTE] paso 16.5 · llevada al 16.5 · [crossTF control_failed, crossTF unusable,
crossmarket DESCARTAR, decay DUDOSA, edgeCost MANTENER, feedQuality MANTENER, gate MANTENER,
snoopingScreen MANTENER, spread MANTENER] · 032522d4cede...
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank
Results --identity 032522d4cedef71ce06e3dbeaff3461a939f965ebb2530ea2999c88fcfb782b --step
16.5 --family donchianUpperCrossUp
  [SUPERVIVIENTE] paso 8 · edgeCost · MANTENER · 003863ecc1d8...
    python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_H1 --databank
Results --identity 003863ecc1d85d6b2f7b71d91be63f495c736fd80bbac5689693dfd4a4cbef45 --step 8 -
-family donchianUpperCrossUp
...
170 supervivientes, 0 dudosas, 0 ya archivadas
descartadas por «DESCARTAR»: 114
descartadas por «control_failed»: 3
descartadas por «fails»: 1
descartadas por «inherited»: 11
descartadas por «unusable»: 1
```

Las 14 primeras líneas son las madres que llegaron al 16.5; las otras 156 son supervivientes del paso 8 que el workflow no llevó más lejos.

Después, para formar el pool se declara con un nombre (un pool declarado queda **congelado**: otro pool es otro nombre). Eso lo hace hoy el motor desde Python (`inputs.pool.declare`); la ventana lo tendrá en la zona PORTFOLIOS.

Qué NO te dice

- **No te dice si una estrategia es buena**: solo que el workflow no la descartó hasta ese paso.
- **No sabe qué es un oos2 disponible**: el archivo guarda hoy build y oos1. Las madres del 16.5 tienen su curva de oos2 en el lote, pero sus operaciones de oos2 aún no existen en ningún export (2026-09-30) — por eso el veredicto de una cartera de desarrollo esperará a que el run termine.
- La etiqueta «paso 8 · edgeCost» nombra el primer estudio del paso: todos los del paso 8 dijeron MANTENER para esa estrategia.

Si algo falla

- `KeyError: 'strategy'` o `'identity'`: un `verdict.csv` antiguo sin esas columnas; se ignora el estudio que no tiene `identity`, pero uno con `identity` y sin `strategy` rompe: dilo.
- Una madre que esperabas no aparece: su lote en `strategyPermutations/<proyecto>/` aún no tiene carpeta `sqx/` (el 16.5 no ha llegado a ella).

78. Cartera · el universo y las parejas — la equity de cada estrategia, y qué parejas pueden ir juntas

Qué pregunta responde

Para un pool declarado: ¿cómo se movió la equity de cada estrategia **cada día y dentro del día** (flotante incluido, minuto a minuto desde M1), en el reloj de cada empresa de fondeo? ¿Cuadra con lo que dice el propio SQX? Y, con eso: ¿qué parejas de estrategias se parecen demasiado para ir en la misma cartera, y por qué filtro cae cada una?

Es el universo (hito M1) y el cribado de parejas (hito M2) del motor de carteras. No elige cartera todavía: eso es la búsqueda (F2 para fondeo), que espera a este paso.

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool declarado (capítulo 77) y quieras ver si sus estrategias se pueden combinar. **No** lo uses para juzgar una estrategia sola: el flotante y la reconciliación son solo el punto de partida.

Antes de empezar

- Un pool declarado en `AlgoData/portfolio/pools/<nombre>.csv`.
- Las barras M1 del feed de cada estrategia en `AlgoData/bars/` (USDJPY y XAUUSD están; los índices no, los importarás cuando haga falta).
- No toca SQX. Lee un fichero de SQX solo para saber el huso de cada feed, en modo lectura.

Cómo se ejecuta

```
python3 -m portfolio.common.construct.universe --pool m1_test_uno
```

flag	obligatorio	qué hace
<code>--pool</code>	sí	el nombre del pool declarado
<code>--set</code>	no	cambia un ajuste de <code>portfolio/common/construct/config.yaml</code> , p. ej. <code>--set pairs.pearson=0.45</code> . Un umbral distinto del tuyo marca el run como AJUSTADO en la primera línea

Tarda unos 7 s por estrategia (lee el M1 de cada feed una sola vez) y usa ~2,6 GB de RAM. La segunda vez con el mismo pool y la misma configuración lee la caché y tarda nada.

Qué produce

Una carpeta por pool y configuración en `AlgoData/portfolio/universe/<hash del pool>-<hash de la config>/` (una caché: se puede borrar, se rehace):

fichero	qué es
daily.parquet	P&L diario a mercado de cada estrategia, en EET (tu reloj común), 0 dentro de su historia y vacío fuera
monthly.parquet	lo mismo por mes
days_ftmo.parquet , days_hantec.parquet	por día del servidor de cada empresa: lo cerrado, el flotante al cierre, el peor y el mejor momento del día
m5_ftmo.npz , m5_hantec.npz	el peor y el mejor flotante de cada bloque de 5 minutos, para sumar estrategias sin perder cuándo coinciden
reconcile.csv	cuánto cuadra cada estrategia con SQX
pairs.parquet	cada medida de cada pareja, solo sobre build
manifest.json	qué pool, qué config, qué relojes, qué versión del código

Cómo se lee el resultado

La **reconciliación** es la licencia para usar el flotante reconstruido. Dos pruebas contra SQX:

- **daily-low**: la curva diaria de SQX es, cada día, la equity **más baja** de ese día (lo cerrado más las posiciones abiertas en su peor mecha). El motor la reconstruye desde M1 y debe coincidir a menos de 1 \$ en al menos el 99 % de los días. En la estrategia archivada coincide el 100 % (máx. 0,16 \$).
- **MAE / MFE**: el peor y el mejor flotante de cada operación, contra las columnas **MAE (\$)** y **MFE (\$)** de SQX. En la archivada, 100 % y 99,9 %.

Una estrategia que no cuadra sale como **excluida**, con su motivo, y no entra en nada: nunca se usa en silencio.

Relojes sin confirmar: el del servidor de Hantec (Nueva York + 7 h) lo diste tú, sin confirmar; hasta leerlo del terminal MT5 de Hantec, todo lo de Hantec lo dice.

Las parejas (solo build, tus umbrales de 0,30):

- Pearson y Spearman, **mensual y diario**: cae si $|\rho| > 0,30$ en cualquiera.
- Co-loss (meses en que pierden las dos) y cola: caen si pasan de 0,30 **por arriba** (la cola entre dos estrategias independientes da $-0,46$ por cómo se eligen sus meses; por eso solo cuenta el lado positivo).
- Ventanas móviles de 60 meses (todo el build y las que acaban en sus últimos 36 meses): cae si el máximo $|\rho|$ pasa de 0,30. Con menos de 60 meses compartidos no se aplican, y se cuenta.
- **Menos de 24 meses compartidos: fuera**, diga lo que diga el coeficiente.
- La correlación en días de estrés (el peor 5 % de días del pool) y el N efectivo **solo se informan**.

Un ejemplo completo

Salida real del 2026-09-30, pool de una sola estrategia (la archivada de USDJPY M30):

```
Universo del pool 'm1_test_uno' (c6a56a636ba5...)
Miembros: 1 conservados, 0 excluidos de 1
  4d679e0c2ce2... [conservada] daily-low build 100%, oos1 100% · MAE 100% · MFE 100%
Calendario: build 2008-01-01...2017-12-31, oos1 2018-01-01...2022-12-31, oos2 2023-01-01...2026-08-30
Relojes sin confirmar: ['hantec']
Carpeta:
/home/sergioguslw/Desktop/AlgoData/portfolio/universe/c6a56a636ba5d09b44a7dd1bb9ffa74c149fc4846463e6ca9ebfae09d2b5dc0b-07f85832e838bf0d
Tiempo: 6.81 s · RAM pico: 2631 MB
```

Con una sola estrategia no hay parejas. Con varias, debajo sale el cribado; así se lee en un pool de prueba de tres, dos de ellas casi clones:

```
Parejas (solo build): 2 admisibles de 3; 3 de 3 estrategias tienen al menos una pareja
admisibles
  caen por pearson_monthly: 1
  caen por pearson_daily: 1
  caen por spearman_monthly: 1
  ...
N efectivo: calma 1.9 / 1.8, estrés 1.5 / 4.0 (autovalores / media de p)
```

Qué NO te dice

- **Que dos estrategias independientes pasen:** con tus umbrales, solo el 75 % de las parejas de estrategias **independientes** pasan todos los filtros sobre 10 años de build (las ventanas móviles y el co-loss rechazan por azar). Para K estrategias independientes, que pasen todas sus parejas es $0,75$ elevado al número de parejas: $4 \rightarrow 18 \%$, $6 \rightarrow 1,5 \%$. Una cartera pequeña puede ser ruido de los filtros, no redundancia.
- **El swap de las noches:** SQX lo cobra al cerrar; el flotante de las noches no lo lleva.
- **El N efectivo por «media de p»** puede pasar de K cuando las correlaciones son negativas (4,0 con 3 estrategias arriba): léelo junto al de autovalores.
- Nada de oos1 ni oos2: las parejas se miden solo en build.

Si algo falla

- `excluida · motivo: ...`: esa estrategia no cuadra con SQX; mírala antes de usarla.
- Un pool que no existe: `FileNotFoundError` con la ruta del CSV.
- Falta el M1 de un feed: `FileNotFoundError` en `AlgoData/bars/<feed>/M1.parquet`.

79. Cartera · la búsqueda de fondeo — qué cartera tiene más probabilidad de aprobar un plan

Qué pregunta responde

Para un plan de fondeo concreto (p. ej. Hantec Enhanced de 10k): de todas las carteras que el cribado de parejas permite, ¿cuál aprueba el challenge con más probabilidad en **6 meses** si la hubieras comprado cualquier día del build, y a qué riesgo por operación? Es la etapa A del camino de fondeo (hito F2 de `portfolio/PLAN.md`): elige candidatas mirando solo build. El juicio fuera de muestra (oos1 + oos2) y el Monte Carlo vienen después (F3, F4).

Cuándo lo usas, y cuándo no

Úsalo cuando tengas un pool cuyas estrategias llevan ya el **stop del paso 24** injertado. **No** te dice si la cartera es buena fuera de muestra, ni si el plan te sale rentable: eso es F4 y la economía del banco (F5).

Antes de empezar

- Un pool declarado (capítulo 77) cuyos miembros sean las versiones **con stop** del paso 24 (SL = $X \cdot \text{ATR}(20)$, $X = p90$), archivadas con su propio retest. Una estrategia sin ese stop queda **fuera** y se dice.
- Nada de SQX. Usa todos los núcleos de la máquina y hasta 80 GB de RAM: **no lo lances mientras el custodio corre un trabajo largo** (los dos no caben juntos).

Cómo se ejecuta

```
python3 -m portfolio.common.construct.funded --pool mi_pool --plan hantec:enhanced:10000:USD
```

flag	obligatorio	qué hace
<code>--pool</code>	sí	el pool declarado
<code>--plan</code>	sí	el plan, con la clave del catálogo (<code>python3 -m portfolio.funded.catalog.show hantec</code>)
<code>--set</code>	no	cambia un ajuste de <code>config.yaml</code> , p. ej. <code>--set funded.horizon_days=63</code>

Qué produce

`AlgoData/portfolio/runs/<fecha>_<pool>_<plan>/` : `search.parquet` (cada cartera evaluada, con sus miembros, su P(aprobar), su riesgo y de dónde salió) y `manifest.json` . Además, **filas en el ledger**: una en el estudio `PORTFOLIO_<pool>_construct` y una en el estudio de cada miembro, con todas las carteras probadas — cada una cuenta como un intento para el Sharpe deflactado.

Cómo se lee el resultado

Las reglas son las tuyas (2026-09-30):

- **Riesgo fijo:** cada operación arriesga r % del saldo **inicial** con su stop $X \cdot \text{ATR}(20)$. SQX simuló con 1.000 \$ sobre $4 \cdot \text{ATR}(20)$; el motor lo convierte con una constante por estrategia.
- **Rejilla:** $r = 0,1 \%, 0,2 \% \dots 1,0 \%$, igual para todas las estrategias de la cartera. Un nivel en el que alguna operación quedaría por debajo de 0,01 lotes no es factible.
- **P(aprobar):** de todos los días de build en los que podrías haber comprado (con 6 meses por delante), qué parte aprueba todas las fases en ≤ 126 días de trading. Las reglas se miran sobre la equity **flotante** del día del servidor de la empresa, con el peor momento del día desde M1.
- **Mismo riesgo en todas las fases** aquí; el riesgo por fase se barre en la etapa B.

La salida lista las 10 mejores: **K** (cuántas estrategias), **P(pass)**, **r** y quiénes son. Una regla sin confirmar del catálogo o el reloj de Hantec salen nombrados.

Un ejemplo completo

Salida real del 2026-09-30 con el pool de prueba: su única estrategia no tiene el stop del paso 24, así que el comando se para sin buscar nada.

```
Pool 'm1_test_uno' plan hantec:enhanced:10000:USD: 1 miembros en el universo
  excluida 4d679e0c2ce2...: sin stop del paso 24
Ningun miembro utilizable: no hay stop del paso 24 en ninguno.
```

La primera búsqueda real espera a que el run H1 llegue al paso 24 y archives sus versiones con stop. Rendimiento medido en datos sintéticos (50 estrategias, 10 años): ~1.500 carteras por segundo en 48 núcleos, así que una búsqueda de 30.000 carteras tarda unos 20 s por plan.

Qué NO te dice

- **Nada fuera de muestra:** elige en build; P(aprobar) en build está inflada por la propia búsqueda.
- **No hay Monte Carlo** todavía: es el camino real de la historia desde cada día de inicio.
- **El redondeo de lotes** no se simula aquí (solo se comprueba que ninguno quede en cero).
- **Una cartera repetida por el genético cuenta dos veces** en el ledger: el número de intentos sale por exceso, nunca por defecto.

Si algo falla

- **sin stop del paso 24**: ese miembro es la versión original; archiva la versión con stop.
- **NotImplementedError** con **min_days**: el catálogo no sabe los días mínimos de ese plan; el fundingWatcher o tú tenéis que confirmarlo.